

# B2B Distribution Problem

Texas Tech University - Computer Science 3383

Adrian De Souza, Daniel Marin and Andy Sanchez

May 2026

## Problem Description

A mid-sized industrial company operates across multiple regions and needs to determine where to build or lease storage and distribution facilities to service all client business while incurring the minimum operational cost possible for the business. The company has a fixed set of client sites (factories, plants & distributors) that require regular shipments of raw materials and finished goods. There is also a fixed set of candidate sites where the company can build or lease storage facility locations each of which can serve clients within a defined coverage radius. The core question is:

What is the minimum number of storage facilities required to cover all client sites, and where are they located in order to minimize the total operational cost?

The core idea behind this problem is to find an optimal solution to a real world problem faced by distribution companies like logistic providers, pharmaceutical companies, and industrial part suppliers. Solving it optimally determines both capital expenditure and long-term operational efficiency required to meet client demand while reducing operational costs.

## Inputs

The problem takes the following inputs:

| Input                          | Description                                                                                                                                                                                                                     |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Client Sites - $C$             | A set of client sites: $C = \{c_1, c_2, \dots, c_n\}$ where each $c_i$ represents a client site with specific coordinates (latitude and longitude).                                                                             |
| Candidate Facility Sites - $F$ | A set of candidate facility sites: $F = \{f_1, f_2, \dots, f_m\}$ where each $f_j$ represents a potential location for a storage facility with specific coordinates (latitude and longitude), and annual operating cost $o_j$ . |
| Budget - $B$                   | The total budget available for building or leasing storage facilities.                                                                                                                                                          |
| Reach Radius - $R$             | The maximum distance (in kilometers) that a storage facility can effectively serve client sites.                                                                                                                                |

## Outputs

A subset of the candidate facility sites is selected to minimize the total operational cost while covering all client sites. The problem requires the following outputs:

| Output                                  | Description                                                                                                         |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Selected Facility Sites - $\mathcal{S}$ | A subset of candidate facility sites: $S \subseteq F$ that are selected for building or leasing storage facilities. |
| Total Operational Cost - $C$            | The total operational cost incurred by the selected facilities, calculated as $C = \sum_{f_j \in S} o_j$ .          |

## Formal Problem Definition

Given the inputs defined above, the B2B Distribution Problem can be formally stated as the following optimization problem:

$$\min_{S \subseteq F} \sum_{f_j \in S} o_j$$

Subject to the following constraints:

- **Coverage constraint:**  $\forall c_i \in C, \exists f_j \in S : d(c_i, f_j) \leq R$ , where  $d(\cdot, \cdot)$  is the Euclidean distance between two coordinate points.
- **Budget constraint:**  $\sum_{f_j \in S} o_j \leq B$
- **Selection constraint:**  $S \subseteq F$

In other words, we seek the least-cost subset of candidate facilities that (a) places every client site within the reach radius of at least one selected facility, and (b) keeps the total operating cost within the available budget.

## NP-Hardness: Reduction from Set Cover

The B2B Distribution Problem is NP-hard. We establish this via a polynomial-time reduction from the classic NP-complete *Set Cover* problem. Informally, the hardness arises because selecting the cheapest covering subset forces a search over all  $2^m$  candidate subsets, with no known polynomial-time shortcut, leading to combinatorial explosion as the number of facilities grows.

### The Set Cover Problem

Given a universe  $U = \{u_1, u_2, \dots, u_n\}$  of  $n$  elements, a collection of subsets  $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$  with  $S_j \subseteq U$ , and a positive integer  $k$ , the Set Cover decision problem asks: does there exist a sub-collection  $\mathcal{S}' \subseteq \mathcal{S}$  with  $|\mathcal{S}'| \leq k$  such that  $\bigcup_{S_j \in \mathcal{S}'} S_j = U$ ? This problem is NP-complete.

### Set Cover $\leq_p$ B2B Distribution

Given an arbitrary Set Cover instance  $(U, \mathcal{S}, k)$ , we construct a B2B instance in polynomial time as follows:  
**Instance transformation:**

1. For each element  $u_i \in U$ , create a client site  $c_i$  placed at coordinates  $(i, 0)$  in a 2D plane.
2. For each subset  $S_j \in \mathcal{S}$ , create a candidate facility  $f_j$  with uniform operational cost  $o_j = 1$ , positioned at the centroid of the client sites  $\{c_i : u_i \in S_j\}$ .
3. Set the reach radius  $R$  large enough so that facility  $f_j$  covers client  $c_i$  if and only if  $u_i \in S_j$ .
4. Set the budget  $B = k$ .

**Correctness:** The B2B instance has a feasible solution of cost  $\leq B = k$  if and only if the Set Cover instance has a cover of size  $\leq k$ .

- ( $\Rightarrow$ ) If a set cover  $\mathcal{S}' \subseteq \mathcal{S}$  of size  $\leq k$  covers all of  $U$ , the corresponding facilities  $\{f_j : S_j \in \mathcal{S}'\}$  cover all client sites in the B2B instance. Since each  $o_j = 1$ , the total cost equals  $|\mathcal{S}'| \leq k = B$ , satisfying both the coverage and budget constraints.
- ( $\Leftarrow$ ) If a B2B solution  $S^* \subseteq F$  of cost  $\leq B = k$  covers all clients, then  $|S^*| \leq k$  (since each  $o_j = 1$ ). The corresponding subsets  $\{S_j : f_j \in S^*\}$  therefore form a set cover of size  $\leq k$  for  $U$ .

This transformation runs in polynomial time in  $|U|$  and  $|\mathcal{S}|$ . Therefore, Set Cover  $\leq_p$  B2B Distribution, which implies the B2B Distribution Problem is NP-hard.

## Methodology

To solve the B2B Distribution Problem, we will implement two algorithmic approaches: an exact brute-force approach and a greedy approximation algorithm. The exact brute-force approach will guarantee finding the optimal solution but may be computationally infeasible for larger instances of the problem due to its exponential time complexity. On the other hand, the greedy approximation algorithm will provide a solution that is not guaranteed to be optimal but can be computed efficiently, making it suitable for larger instances of the problem. We will analyze the time complexities of both approaches and compare their performance on relatively small test cases to evaluate their effectiveness in solving the B2B Distribution Problem.

This will allow us to understand the trade-offs between optimality and computational efficiency in solving this problem, and to identify scenarios where one approach may be more suitable than the other based on the size and complexity of the input data. Placing emphasis on how at scale approximations and randomized algorithms are often necessary to find good solutions to NP-hard problems in a reasonable amount of time, while exact algorithms may only be feasible for small instances or when computational resources are abundant.

## Where to Find the Code

The code can be found in the `source` directory of the repository, under the GitHub Repository. The exact brute-force approach is implemented in the file `brute-force.py`, while the greedy approximation algorithm is implemented in the file `greedy.py`. Both implementations include functions to read input data, compute coverage, and evaluate solutions based on the defined constraints of the problem. Additionally, the `benchmark.py` file contains code to generate test instances of the problem and to compare the performance of both algorithms across a range of scenarios. The implementations are designed to be modular and well-documented, allowing for easy understanding and potential modifications for

## Algorithmic Approaches

Throughout this section, we will discuss two algorithmic approaches to solve the B2B Distribution Problem, the first being an exact brute-force approach which is guaranteed to find the optimal cheapest solution, and the second being a greedy approximation algorithm which is tasked with leveraging a heuristic to find a solution that is not guaranteed to be optimal, but is computationally efficient and can be used for larger instances of the problem. Following the discussion of these two approaches, we will analyze their time complexities and compare their performance on various test cases.

### Exact Brute-Force Approach

The exact brute-force approach to solving the B2B Distribution Problem involves enumerating all possible subsets of candidate facility sites and checking which subsets cover all client sites while minimizing the total operational cost. The algorithm is found in the file `brute-force.py` and can be summarized as follows:

**Step 1.** Generate all possible subsets of candidate facility sites.

- Step 2.** For each subset, check if its total sum of operational costs is within the budget, and if it is less than the current minimum cost found. This is a preemptive check to avoid unnecessary coverage checks for subsets that are already too expensive thus saving time.
- Step 3.** If the subset is within budget, check if all client sites are covered by the facilities in the subset. This is done by using a precomputed coverage matrix that indicates which client sites are covered by which candidate facility sites based on the reach radius.
- Step 4.** If all client sites are covered, update the minimum cost and the corresponding subset of facilities if the cost is lower than the previously recorded minimum cost.

### Time Complexity Analysis

The time complexity of the exact brute-force approach is exponential in the number of candidate facility sites, as it requires checking all possible subsets of candidate facilities. Specifically, if there are  $m$  candidate facility sites, there are  $2^m$  possible subsets to evaluate. For each subset, we need to check if it covers all client sites, which can be done in  $O(n)$  time where  $n$  is the number of client sites. Therefore, the overall time complexity of the brute-force approach is  $\Theta(n \cdot 2^m)$ , making it impractical for large instances of the problem.

### Greedy Approximation Algorithm

The greedy approximation algorithm for the B2B Distribution Problem is designed to find a solution that is not guaranteed to be optimal but can be computed efficiently. The algorithm is found in the file `greedy.py` and can be summarized as follows:

- Step 1.** Initialize an empty set of selected facilities and a set of uncovered client sites.
- Step 2.** While there are still uncovered client sites:
- a. For each candidate facility site, calculate the cost-effectiveness ratio, which is defined as the number of uncovered client sites that would be covered by selecting that facility divided by its operational cost.
  - b. Select the candidate facility site with the highest cost-effectiveness ratio and add it to the set of selected facilities.
  - c. Update the set of uncovered client sites by removing those that are now covered by the newly selected facility.

This is the conventional greedy approach to the Set Cover Problem, which is known to have a logarithmic approximation ratio. In the context of the B2B Distribution Problem, this means that the cost of the solution found by the greedy algorithm will be at most  $O(\log n)$  times the cost of the optimal solution, where  $n$  is the number of client sites. This is because the greedy algorithm prioritizes facilities that cover the most uncovered client sites per unit of cost, which is a common heuristic for set cover problems; placing emphasis on cost-effectiveness helps to ensure that the algorithm makes efficient choices in terms of coverage and cost despite not guaranteeing an optimal solution.

### Time Complexity Analysis

The time complexity of the greedy approximation algorithm is polynomial in the number of candidate facility sites and client sites. Specifically, in each iteration, we need to calculate the cost-effectiveness ratio for each candidate facility site, which takes  $O(m \cdot n)$  time where  $m$  is the number of candidate facilities and  $n$  is the number of client sites. Since we may need to select up to  $m$  facilities in the worst case, the overall time complexity of the greedy approximation algorithm is  $\Theta(m^2 \cdot n)$ , which is significantly more efficient than the exponential time complexity of the brute-force approach.

## Performance Comparison

To compare the performance of the brute-force and greedy algorithms, we ran both algorithms on a set of test cases with constant client sites (as the number of client sites does not affect the exponential growth of the brute-force algorithm) and varying numbers of candidate facility sites, while keeping the budget and reach radius constant to ensure a fair comparison. The comparison variables ended up being:

- Clients per instance - 12
- Facilities per instance ( $m$ ) - from 8 to 18
- Trials per  $m$  - 5
- Radius - 200 km
- Budget Fraction - 0.5 of the total facility cost
- Base Seed - 2026 (for instance generation)

While performing the benchmarking we tracked overall execution time, cost ratio (the cost of the solution found by the greedy algorithm divided by the cost of the optimal solution found by the brute-force algorithm), and measuring the feasibility of the solutions (whether they meet the budget and coverage requirements, which is entirely dependent on the specific instance of the problem). The results of the performance comparison are summarized in the following table:

| <b>m</b> | <b>Greedy Avg</b> | <b>BF Avg</b> | <b>Speedup</b>  | <b>Problem Feasibility</b> | <b>Greedy Cost Ratio</b> |
|----------|-------------------|---------------|-----------------|----------------------------|--------------------------|
| 8        | 11.6 $\mu$ s      | 0.122ms       | 10.6 $\times$   | 0.4                        | 1.0549                   |
| 9        | 11.4 $\mu$ s      | 0.217ms       | 19.1 $\times$   | 0.6                        | 1.0363                   |
| 10       | 11.0 $\mu$ s      | 0.446ms       | 41.0 $\times$   | 0.6                        | 1.0349                   |
| 11       | 14.0 $\mu$ s      | 0.951ms       | 68.7 $\times$   | 0.6                        | 1.0295                   |
| 12       | 14.0 $\mu$ s      | 1.534ms       | 112.0 $\times$  | 1.0                        | 1.1400                   |
| 13       | 16.0 $\mu$ s      | 3.224ms       | 205.3 $\times$  | 1.0                        | 1.1255                   |
| 14       | 24.0 $\mu$ s      | 6.500ms       | 268.4 $\times$  | 1.0                        | 1.0426                   |
| 15       | 23.0 $\mu$ s      | 14.022ms      | 609.1 $\times$  | 1.0                        | 1.1110                   |
| 16       | 23.0 $\mu$ s      | 25.461ms      | 1107.0 $\times$ | 1.0                        | 1.0286                   |
| 17       | 22.0 $\mu$ s      | 58.162ms      | 2617.5 $\times$ | 0.8                        | 1.0560                   |
| 18       | 24.0 $\mu$ s      | 117.202ms     | 4936.9 $\times$ | 0.8                        | 1.0449                   |

Table 1: Performance Comparison of Brute-Force and Greedy Algorithms, where  $m$  is the number of candidate facility sites, note that feasibility isn't guaranteed; some problems simply might not have a solution for the configuration still computation takes place.

While performing the comparisons we managed to observe that the greedy algorithm consistently outperformed the brute-force algorithm in terms of execution time, with speedups ranging from approximately 10 $\times$  for smaller instances to over 4900 $\times$  for larger instances. The cost ratio of the greedy solution compared to the optimal solution found by the brute-force algorithm was generally close to 1 (with a  $\pm 0.05 - 0.11$  for the given subset), indicating that the greedy algorithm was able to find solutions that were near-optimal in terms of cost and was also capable of finding the optimal solution in 52.3% of the instances. However, the feasibility of the solutions varied across different instances, with some instances being infeasible for both algorithms due to the constraints of the problem (e.g., budget and coverage requirements). Overall, the greedy approximation algorithm provided a significant improvement in computational efficiency while still yielding solutions that were close to optimal in terms of cost, making it a practical choice for larger instances of the B2B Distribution Problem where the brute-force approach becomes computationally infeasible.

## Infeasibility Threshold of the Exact Algorithm

The benchmark data reveals a consistent doubling of brute-force runtime with each unit increase in  $m$ , matching the theoretical  $\Theta(n \cdot 2^m)$  prediction. From  $m = 8$  to  $m = 18$ , the runtime grew from  $0.122ms$  to  $117.202ms$ , a factor of approximately  $961\times$  over 10 steps, closely matching the expected  $2^{10} = 1024$ . Extrapolating this doubling trend yields the following estimated runtimes:

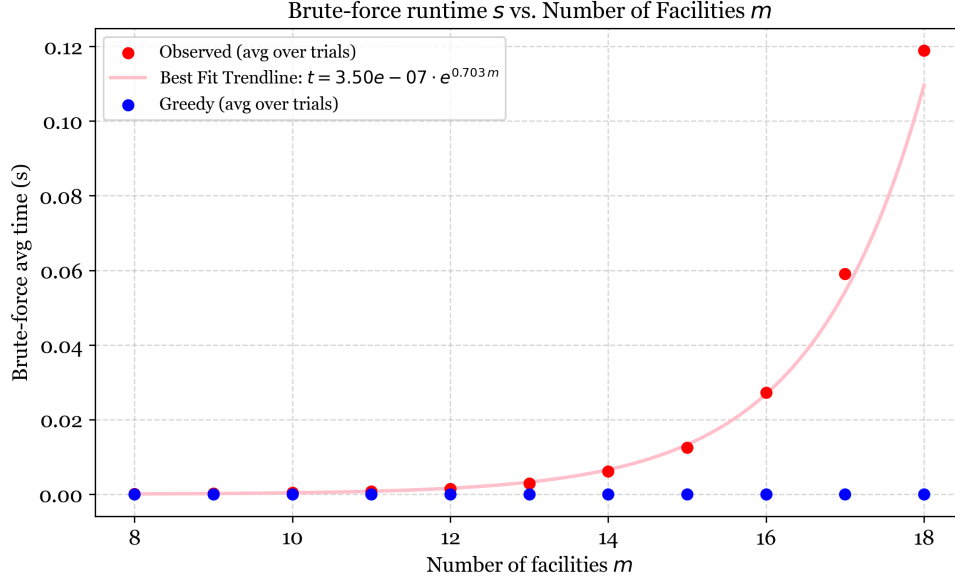


Figure 1: Runtimes for the Brute-Force Algorithm as  $m$  Increases, based on the observed doubling trend in the benchmark data.

Based on this analysis, the brute-force algorithm becomes effectively infeasible at approximately  $m \approx 25$ , where runtimes begin to exceed several seconds per instance. Beyond  $m \approx 30$ , the algorithm is entirely impractical for any real-world use. This behavior is expected for NP-hard problems: since no polynomial-time exact solver is known, the search space grows exponentially and even modest increases in  $m$  render computation intractable. In contrast, the greedy algorithm remained stable in the range of  $11\text{--}24\mu s$  across all tested values of  $m$ , confirming its practical scalability well beyond the brute-force infeasibility threshold.

Note that the performance comparison and the overall results will vary based on computer specs, the specific instances of the problem generated, and the random seed used for instance generation. The results presented here are based on a specific set of instances and may not generalize to all possible instances of the B2B Distribution Problem, potentially leading to different cost ratios and feasibility outcomes in other scenarios.

## Conclusion

Overall, the B2B Distribution Problem is a real-world problem that is NP-hard due to its nature of optimization that directly relates to the Set Cover with a cost component. The exact brute-force approach guarantees finding the optimal solution but is computationally infeasible for larger instances due to its exponential time complexity. In contrast, the greedy approximation algorithm provides a practical near optimal solution with significantly improved computational efficiency, making it suitable for larger instances of the problem. Overall the trade-off between optimality and efficiency is evident in the performance comparison, where greedy algorithms can yield solutions that are close to optimal while being computationally feasible, especially as the size of the problem increases. This highlights the importance of approximation algorithms in solving NP-hard problems in real-world scenarios where exact solutions may not be practical.